

Vivaz y Polo — guía rápida de API

Dominio público: <https://yen-entrench-grader.ngrok-free.dev>

Vivaz y Polo comparten este dominio. Vivaz usa la raíz; las rutas de Polo comienzan con `/polo`.

```
export VIVAZ_BASE_URL="https://yen-entrench-grader.ngrok-free.dev"
export POLO_BASE_URL="${VIVAZ_BASE_URL}/polo"
```

Vivaz — LLM Gateway

Las rutas `/health` y `/ready` sirven para comprobar disponibilidad. No requieren API key.

```
curl -sS -H 'ngrok-skip-browser-warning: 1' \
  "$VIVAZ_BASE_URL/health"

curl -sS -H 'ngrok-skip-browser-warning: 1' \
  "$VIVAZ_BASE_URL/ready"
```

Para consultar modelos y generar respuestas, se requiere `Authorization: Bearer ....`. Configura `VIVAZ_API_KEY` de forma privada en tu aplicación o secret manager; no incluyas el valor en esta guía ni en el código compartido.

Modelos disponibles

```
curl -sS -H 'ngrok-skip-browser-warning: 1' \
  -H "Authorization: Bearer ${VIVAZ_API_KEY}" \
  "$VIVAZ_BASE_URL/v1/models"
```

Chat

```
curl -sS -X POST \
  -H 'ngrok-skip-browser-warning: 1' \
  -H "Authorization: Bearer ${VIVAZ_API_KEY}" \
  -H 'Content-Type: application/json' \
  "$VIVAZ_BASE_URL/v1/chat/completions" \
  -d '{
    "messages": [
      {"role": "user", "content": "Resume en una oración qué es ngrok."}
    ],
    "temperature": 0.3,
    "max_tokens": 128,
```

```
"think": false
}'
```

La respuesta incluye `choices[0].message.content` y el uso de tokens en `usage`. Este endpoint devuelve una respuesta JSON completa; no acepta `stream`.

Después de cada solicitud, el modelo permanece cargado hasta 5 minutos sin actividad y luego se descarga de la GPU. La siguiente solicitud lo vuelve a cargar automáticamente; la primera respuesta tras el período inactivo puede tardar más.

Polo — proxy de Ollama con autenticación

Todas las rutas de Polo requieren su propio token Bearer. `POLO_API_TOKEN` es distinto de `VIVAZ_API_KEY`; guárdalo de forma privada y no lo compartas.

Modelos instalados

```
curl -sS -H 'ngrok-skip-browser-warning: 1' \
-H "Authorization: Bearer ${POLO_API_TOKEN}" \
"$POLO_BASE_URL/api/tags"
```

Chat nativo de Ollama

```
curl -sS -X POST \
-H 'ngrok-skip-browser-warning: 1' \
-H "Authorization: Bearer ${POLO_API_TOKEN}" \
-H 'Content-Type: application/json' \
"$POLO_BASE_URL/api/chat" \
-d '{
  "model": "qwen3.5:9b",
  "messages": [
    {"role": "user", "content": "Hola, responde brevemente."}
  ],
  "stream": false
}'
```

Polo conserva el formato nativo de Ollama. Puedes usar otras rutas compatibles, como `/api/generate`; el prefijo `/polo` se elimina antes de reenviar la solicitud.

El proxy fija `keep_alive` en 5 minutos para las rutas de inferencia (`/api/chat`, `/api/generate`, `/api/embed` y `/api/embeddings`), incluso si el cliente envía otro valor. Tras 5 minutos sin solicitudes Ollama descarga el modelo; la siguiente solicitud lo vuelve a cargar automáticamente.

Autenticación y errores comunes

Respuesta	Significado
200	Solicitud correcta
401	Falta el Bearer token o no es válido
503	El servicio local de destino no está disponible
ERR_NGROK_3200	El servidor o el agente ngrok está apagado/desconectado

El hostname se mantiene fijo al reiniciar el agente. Mientras el servidor esté apagado, las APIs no estarán disponibles; systemd inicia el router y ngrok cuando el servidor arranca.

El header `ngrok-skip-browser-warning: 1` ayuda a evitar la pantalla intermedia de ngrok en clientes que la reciban. No sustituye la autenticación Bearer.